Chapter 2: Data transmission

Types and methods of data transmission

Data packets

- Data sent over long distances is usually broken up into **data packets** called datagrams.
- The packets of data are usually quite small, typically 64KiB,which are much easier to control than a long continuous stream of data.
- The idea of splitting up data in this way means each packet can be sent along a different route to its destination.

Benefit

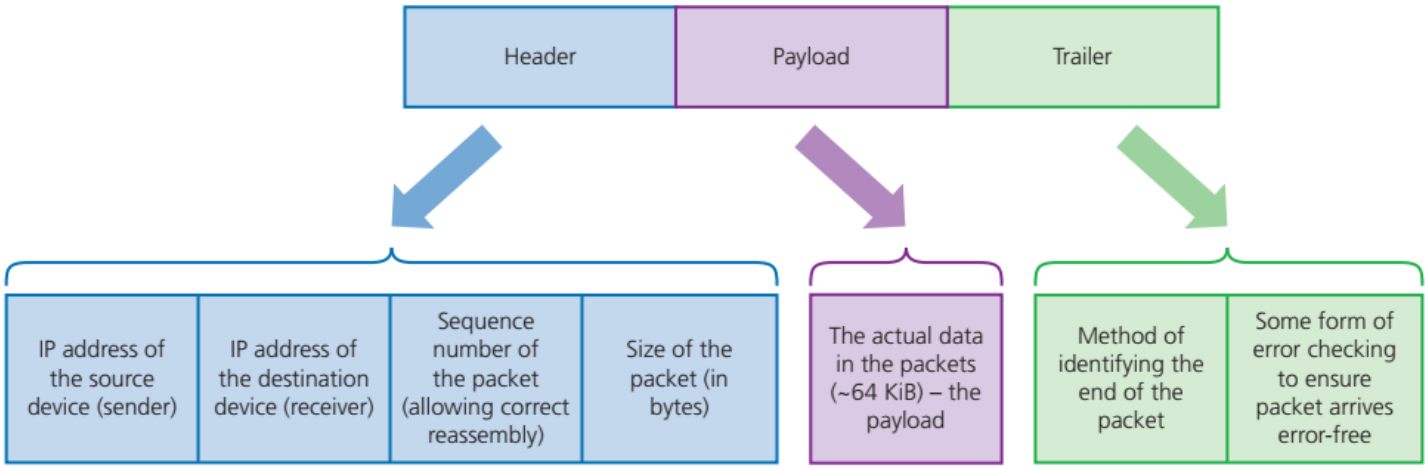
This would clearly be of great benefit if a particular transmission route was out of action or very busy.

Drawback

The only obvious drawback of splitting data into packets is the need to reassemble the data when it reaches its destination.

Packet structure

A typical packet is split up into:
» a packet header
» the payload
» a trailer



▲ **Figure 2.1** Packet structure

For **each** packet, the **packet header** consists of:
» the IP address of the sending device
» the IP address of the receiving device

» the sequence number of the packet (this is to ensure that all the packets can be reassembled into the correct order once they reach the destination)
» packet size (this is to ensure the receiving station can check if all of the packets have arrived intact).
(Note: the header often also contains another value indicating how many packets there are in total for this transmission.)

For **each** packet, the **payload** consists of the actual data being sent in the packet (this is usually about 64KiB).

For **each** packet, the **packet trailer** consists of:
» some way of identifying the end of the packet; this is essential to allow
each packet to be separated from each other as they travel from sending to receiving station
» an error checking method; **cyclic redundancy checks** (CRCs) are used to check data packets:
– this involves the sending computer adding up all the 1-bits in the payload and storing this as a hex value in the trailer before it is sent.
– once the packet arrives, the receiving computer recalculates the number of 1-bits in the payload
– the computer then checks this value against the one sent in the trailer.
– if the two values match, then no transmission errors have occurred;otherwise the packet needs to be re-sent.

Packet switching

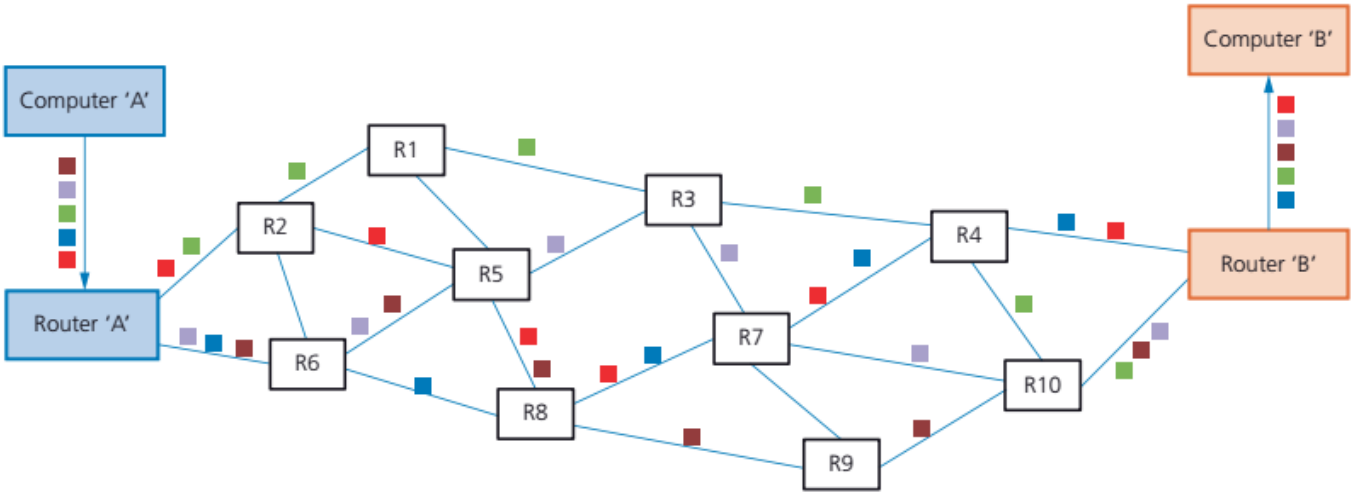
Packet switching is a method of data transmission in which a message is broken up into a number of packets.

Each packet can then be sent independently from start point to end point.

At the destination, the packets will need to be reassembled into their correct order (using the information sent in the header).
At each stage in the transmission, there are **nodes** that contain a router.

Each router will determine which route the packet needs to take, in order to reach its destination (the destination IP address is used in this part of the process).

Suppose our photograph (Figure 2.3) has been split up into five packets that have been sent in the following order:
▲ **Figure 2.3**
» each packet will follow its own path (route)
» routers will determine the route of each packet
» routing selection depends on the number of packets waiting to be processed at each node
» the shortest possible path **available** is always selected – this may not always be the shortest path that **could** be taken, since certain parts of the route may be too busy or not suitable
» unfortunately, packets can reach the destination in a different order to that in which they were sent.

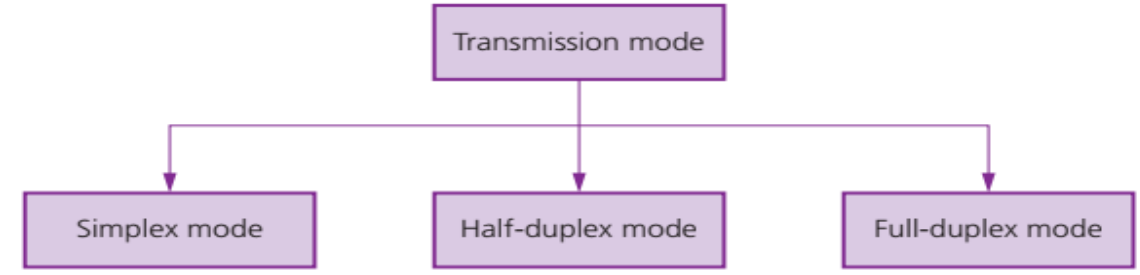


▲ **Figure 2.5** Typical network showing possible paths taken by each packet

Benefits	Drawbacks
There is no need to tie up a single communication line.	Packets can be lost and need to be re-sent.
It is possible to overcome failed, busy or faulty lines by simply re-routing packets	There is a delay at the destination whilst the packets are being re-ordered.
It is relatively easy to expand package usage.	The method is more prone to errors with **real-time streaming** (for example, a live sporting event being transmitted over the internet).
A high data transmission rate is possible.	

Data Packets

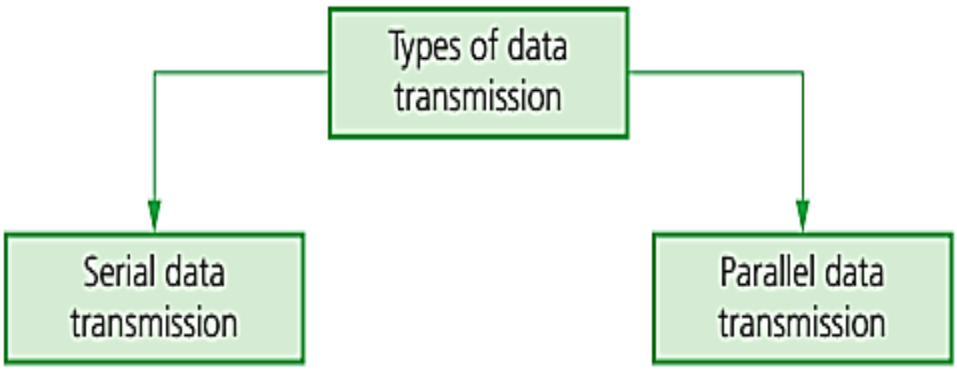
Essentially, three factors need to be considered when transmitting data:
» the direction of data transmission (for example, can data transmit in one direction only, or in both directions)
» the method of transmission (for example, how many bits can be sent at the same time)
» how will data be synchronised (that is, how to make sure the received data is in the correct order)



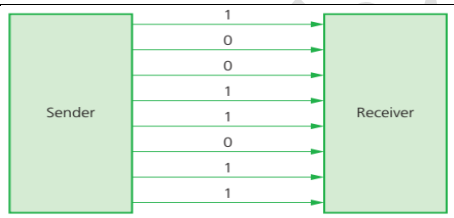
▲ **Figure 2.8** Transmission modes

Simplex Data Transmission	Half-Duplex Data Transmission	Full-Duplex Data Transmission
Simplex mode occurs when data can be sent in **ONE DIRECTION ONLY.**	**Half-duplex** mode occurs when data is sent in **BOTH DIRECTIONS** but **NOT AT THE SAME TIME**	**Full-duplex** mode occurs when data can be sent in **BOTH DIRECTIONS AT THE SAME TIME**
From sender to receiver only.	Data can be sent from 'a' to 'b' and from 'b' to 'a' along the same transmission line, but they can't both be done at the same time.	Data can be sent from 'a' to 'b' and from 'b' to 'a' along the same transmission line simultaneously.
An example of this would be sending data from a computer to a printer.	An example of this would be a walkie-talkie where a message can be sent in one direction only at a time; but messages can be both received and sent.	An example of this would be a broadband internet connection.

Types of Data Transmission

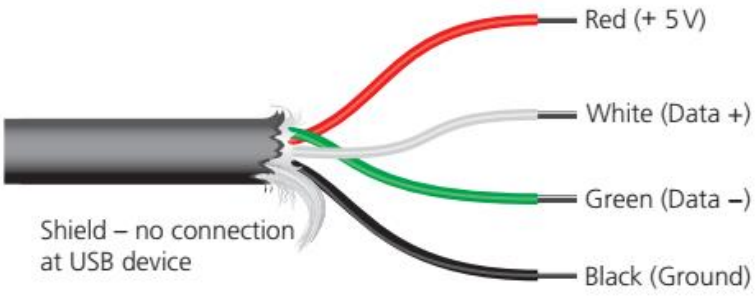


▲ **Figure 2.9** Types of data transmission

Serial Data Transmission	Parallel Data Transmission
Serial data transmission occurs when data is sent **ONE BIT AT A TIME** over a **SINGLE WIRE/CHANNEL**. Bits are sent one after the other as a single stream.	**Parallel data transmission** occurs when **SEVERAL BITS OF DATA** (usually one byte) are sent down **SEVERAL CHANNELS/WIRES** all at the same time. Each channel/wire transmits one bit.
	
Serial data transmission works well over long distances.	Parallel data transmission works well over short distances. (for example, used in internal pathways on computer circuit boards).
Data is transmitted at a slower rate than parallel data transmission.	It is, however, a faster method of data transmission than serial.
Because only one channel/wire is used, data will arrive at its destination fully synchronised (i.e.in the correct order).	Data can become **skewed** (that is, the data can arrive unsynchronised) and bits can arrive out of order.
An example of its use is when connecting a computer to a printer via a USB connection	The internal circuits in a computer use parallel data transmission since the distance travelled between components is very short and high-speed transmission is essential.

Universal serial bus (USB)

- **Universal Serial Bus (USB)** is a form of serial data transmission.
- USB is now the most common type of input/output port found on computers and has led to a standardisation method for the transfer of data between devices and a computer.
- It is important to note that USB allows both half-duplex and full-duplex data transmission.



▲ **Figure 2.12** Typical USB cable

USB cable consists of a four-wired shielded cable, two wires for power (red and black). Other two wires (white and green) are for data transmission.

When a device is plugged into a computer using one of the USB ports:
» the computer automatically detects that a device is present (this is due to a small change in the voltage on the data signal wires in the USB cable).
» the device is automatically recognised, and the appropriate device driver software is loaded up so that the computer and device can communicate effectively
» if a new device is detected, the computer will look for the device driver that matches the device; if this is not available, the user is prompted to download the appropriate driver software (some systems do this automatically and the user will see a notice asking for permission to connect to the device website).

Benefits and drawbacks of USB systems

Benefits	Drawbacks
Devices plugged into the computer are automatically detected and device drivers are automatically loaded up.	Standard USB only supports a maximum cable length of 5 m; beyond that, USB hubs are needed to extend the cable length.
Connections can only fit one way preventing incorrect connections being made.	
It has become an industry standard, which means considerable support is available.	Even though USB is backward compatible, very early USB standards (V1) may not always be supported by the latest computers
Can support different data transmission rates (from 1.5 Mbps to 5 Gbps).	
No need for external power source since cable supplies +5 V power.	Even the latest version 3 (V3) and version 4 (V4) USB-C systems have a data transfer rate which is slow compared to, for example, Ethernet connections (Note: USB V2 has a maximum data transfer rate of 480 Mbps.)
USB protocol notifies the transmitter to re- transmit data if any errors are detected; this leads to error-free data transmission.	
It is relatively easy to add more USB ports if necessary, by using USB hubs	
USB is backward compatible (that is, older versions are still supported).	

Type – C

- USB-C, is now becoming more common in laptops and tablets/phones. This is a 24-pin symmetrical connector which means it will fit into a USB-C port either way round.
- It is much smaller and thinner than older USB connectors, offers 100 watt (20 volt) power connectivity, which means full-sized devices can now be charged and it can carry data at 10 gigabits per second (10 Gbps); this means it can now support 4K video delivery.
- USB-C is backward compatible (to USB 2.0 and 3.0) , and is expected to become the new industry standard (universal) format.

Methods of Error Detection

Errors can occur during data transmission due to:
» interference (all types of cable can suffer from electrical interference, which can cause data to be corrupted or even lost)
» problems during packet switching (this can lead to data loss – or it is even possible to gain data!)
» skewing of data (this occurs during parallel data transmission and can cause data corruption if the bits arrive out of synchronisation).

There are a number of ways data can be checked for errors following transmission:

1. Parity checks

• A parity bit is transmitted with each byte of data Check is performed when data is received.
• Odd or even (parity can be used).
• Counts number of 1's to see if 1's are even(2,4,6,8) or odd(1,3,5,7).
• (Each byte is) checked after transmission to see if it matches the odd/even parity used Check is performed when data is received.

2.Echo check

- A copy of the data is sent back to the sender.
- The returned data is compared with the original data by the sender's computer.
- If there are no differences, then the data was sent without error.
- If the two sets of data are different, then an error occurred at some stage during the data transmission.

3.ARQ

- ARQ uses positive and negative **acknowledgements** (indicating that data has/has not been received correctly) and **timeout.**
- This is used to detect whether the received data contains any transmission errors.
- If no error is detected, a positive acknowledgement is sent back to the sending device.
- However, if an error is detected, the receiving device now sends a negative acknowledgement to the sending device and requests re-transmission of the data.
- A time-out is used by the sending device by waiting a pre-determined amount of time .
- If no acknowledgement of any type has been received by the sending device within this time limit, it automatically re-sends the data until a positive acknowledgement is received .
- ***ARQ is often used by mobile phone networks to guarantee data integrity.***

4.Check Digit

A **check digit** is the final digit included in a code. It is calculated from all the other digits in the code. Check digits are used for barcodes on products, such as International Standard Book Numbers (ISBN) and Vehicle Identification Numbers (VIN).
Check digits are used to identify errors in *data entry* caused by mis-typing or mis-scanning a barcode. They can usually detect the following types of error:

- An **incorrect digit** entered, for example 5327 entered instead of 5307.
- **Transposition** errors where two numbers have changed order, for example 5037 instead of 5307.
- **Omitted** or **extra digits**, for example 537 instead of 5307 or 53107 instead of 5307.
- **Phonetic** errors, for example 13 (thirteen), instead of 30 (thirty).

5.Checksum

- When a block of data is about to be transmitted, the checksum is calculated from the block of data.
- The calculation is done using an agreed algorithm.
- The checksum is then transmitted with the block of data.
- At the receiving end, the checksum is recalculated by the computer using the block of data.
- The re-calculated checksum is then compared to the checksum sent with the data block.
- If the two checksums are the same, then no transmission errors have occurred; otherwise a request is made to re-send the block of data.

Symmetric and Asymmetric Encryption

- A hacker is often referred to as an **eavesdropper**.
- **Eavesdropper** – another name for a hacker who intercepts data being transmitted on a wired or wireless network.
- Using **encryption** helps to minimise this risk.
- **Encryption** – the process of making data meaningless using encryption keys; without the correct decryption key the data cannot be decoded (unscrambled).
- It cannot prevent the data being intercepted, but it stops it from making any sense to the eavesdropper.
- This is particularly important if the data is sensitive or confidential (for example, **credit card/bank details, medical history or legal documents**).

Plaintext and ciphertext

The original data being sent is known as **plaintext**. Once it has gone through an **encryption algorithm**, it produces **ciphertext**:

Plaintext – the original text/message before it is put through an encryption algorithm.
Ciphertext – encrypted data that is the result of putting a plaintext message through an encryption algorithm.

Encryption algorithm – a complex piece of software that takes plaintext and generates an encrypted string known as ciphertext
Quantum computer – a computer that can perform very fast calculations; it can perform calculations that are based on probability rather than simple 0 or 1 values; this gives a quantum computer the potential to process considerably more data than existing computers

Symmetric Encryption

Symmetric encryption uses an encryption key; the same key is used to encrypt and decrypt the encoded message.

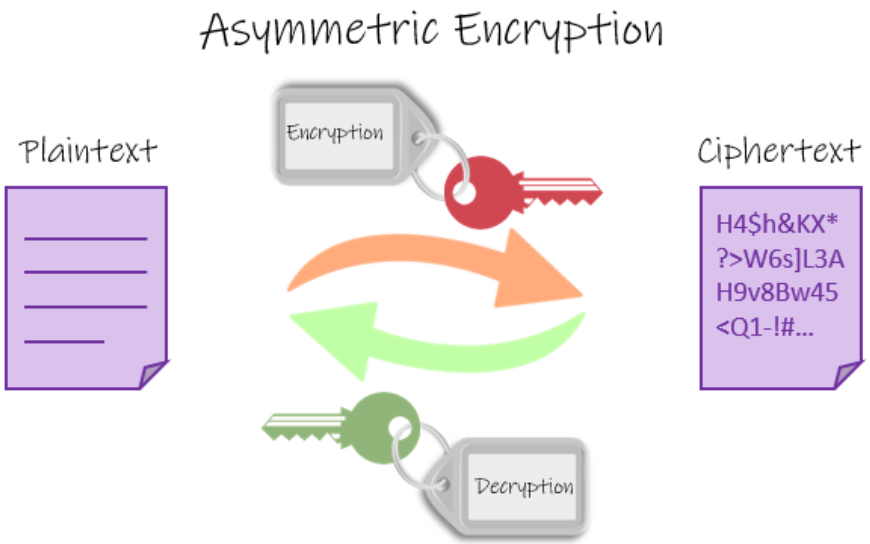
What remains an important issue in Symmetric encryption?

The real difficulty is keeping the encryption key a secret (for example, it needs to be sent in an email or a text message which can be intercepted). Therefore, the issue of security is always the main drawback of symmetrical encryption, since a single encryption key is required for both sender and recipient.

Asymmetric Encryption

Asymmetric encryption was developed to overcome the security problems associated with symmetric encryption. It makes use of two keys called the **public key** and the **private key**:
» **Public** key (made available to everybody)
» **Private** key (only known to the computer user).

With this approach **a pair of linked keys** is used and consists of a **public key**, used to **encrypt** data and a **private key** used to **decrypt** data. Both keys are different (but related). The public key, is available to everyone who wishes to send a message. On the other hand, the private key is kept at a secure place by the owner of the public key. Both types of key are needed to encrypt and decrypt messages.



Past/specimen papers reference

March 2019 Paper 12, question 4(d) and question 6(a)
June 2019 Paper 12, question 9
June 2019 Paper 13, question 3(b) and question 5
Nov 2019 Paper 11, question 5
Nov 2019 Paper 12, question 5(c)(ii), question 9 and question 10(a)
Nov 2019 Paper 13, question 1(b) question 3 and question 5
March 2020 Paper 12, question 2
June 2020 Paper 11, question 3(a), 3(b) and question 4
June 2020 Paper 12, question 3
June 2020 Paper 13, question 9
2023 Specimen Paper 1, question 2(a), 2(b) and question 9